

Estructuras de Datos - Arboles de Expansion Minima

Introduccion

Un **arbol de expansion minima** (MST, por sus siglas en ingles) es un subgrafo de un grafo no dirigido y ponderado que conecta todos los vertices con el costo total minimo. Este concepto es ampliamente utilizado en redes de comunicaciones, sistemas de distribucion y problemas de optimizacion.

Definicion

Dado un grafo $G = (V, E)$:

- G es **no dirigido**.
- Cada arista $(u, v) \in E$ tiene un peso no negativo $w(u, v)$.
- El MST es un grafo parcial conexo que incluye todos los vertices de G y un subconjunto de E , tal que:
 - Es aciclico.
 - La suma de los pesos de sus aristas es minima.

Algoritmos Principales

Existen dos algoritmos principales para construir un arbol de expansion minima:

1. Algoritmo de Prim

El algoritmo de Prim construye el MST seleccionando siempre la arista de menor peso que conecta un vertice en el conjunto actual del MST con un vertice fuera de este.

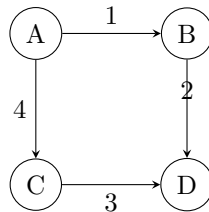
Pasos del Algoritmo:

1. Inicia con un vertice arbitrario y añádalo al conjunto W .
2. Encuentra la arista de menor peso que conecta un vertice en W con un vertice fuera de W .
3. Añade el vertice y la arista seleccionada al MST.

4. Repite hasta que W contenga todos los vertices de V .

Complejidad: $O(V^2)$ en grafos densos o $O(E \log V)$ utilizando un monticulo minimo.

Ejemplo:



2. Algoritmo de Kruskal

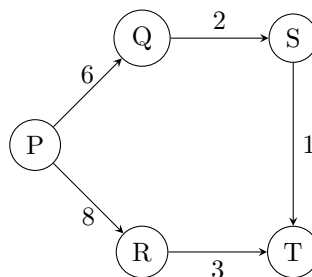
El algoritmo de Kruskal construye el MST seleccionando siempre la arista de menor peso, asegurándose de que no forme ciclos con las aristas ya seleccionadas.

Pasos del Algoritmo:

1. Ordena todas las aristas del grafo por peso.
2. Inicializa un MST vacio.
3. Añade aristas al MST en orden ascendente de peso, siempre que no formen un ciclo.
4. Repite hasta que el MST contenga $V - 1$ aristas.

Complejidad: $O(E \log E)$, ya que la parte mas costosa es ordenar las aristas.

Ejemplo:



Comparacion entre Prim y Kruskal

- Prim es mas eficiente en grafos densos debido a su enfoque basado en vertices.
- Kruskal es mas eficiente en grafos dispersos ya que opera sobre aristas ordenadas.

Implementacion del Algoritmo de Prim

El siguiente codigo implementa el algoritmo de Prim para construir un arbol de expansion minima. Este algoritmo selecciona siempre la arista de menor peso que conecta un vertice del MST con uno fuera de este.

Codigo en Java:

```
1 import java.util.*;
2
3 class Prim {
4     // Metodo para ejecutar el algoritmo de Prim
5     public void primMST(int[][] graph) {
6         int vertices = graph.length;
7         boolean[] mstSet = new boolean[vertices]; // Conjunto de vertices incluidos
8             en el MST
9         int[] key = new int[vertices]; // Costos minimos para llegar a cada vertice
10         int[] parent = new int[vertices]; // Almacena el MST
11
12         Arrays.fill(key, Integer.MAX_VALUE); // Inicializa todas las llaves con
13             infinito
14         key[0] = 0; // El primer vertice tiene llave 0
15         parent[0] = -1; // El primer nodo no tiene padre
16
17         for (int count = 0; count < vertices - 1; count++) {
18             int u = minKey(key, mstSet); // Selecciona el vertice de menor llave
19             mstSet[u] = true; // Incluye el vertice en el MST
20
21             // Actualiza los valores de las llaves para los vertices adyacentes
22             for (int v = 0; v < vertices; v++) {
23                 if (graph[u][v] != 0 && !mstSet[v] && graph[u][v] < key[v]) {
24                     parent[v] = u;
25                     key[v] = graph[u][v];
26                 }
27             }
28             printMST(parent, graph);
29         }
30
31         // Encuentra el vertice con la llave minima
32         private int minKey(int[] key, boolean[] mstSet) {
33             int min = Integer.MAX_VALUE, minIndex = -1;
34             for (int v = 0; v < key.length; v++) {
35                 if (!mstSet[v] && key[v] < min) {
36                     min = key[v];
37                     minIndex = v;
38                 }
39             }
40             return minIndex;
41         }
42
43         // Imprime el MST
44         private void printMST(int[] parent, int[][] graph) {
45             System.out.println("Arista \tPeso");
46             for (int i = 1; i < graph.length; i++) {
47                 System.out.println(parent[i] + " - " + i + "\t" + graph[i][parent[i]]);
48             }
49         }
50     }
51 }
```

```

49
50     public static void main(String[] args) {
51         int[][] graph = {
52             {0, 2, 0, 6, 0},
53             {2, 0, 3, 8, 5},
54             {0, 3, 0, 0, 7},
55             {6, 8, 0, 0, 9},
56             {0, 5, 7, 9, 0}
57         };
58         Prim prim = new Prim();
59         prim.primMST(graph);
60     }
61 }

```

Implementacion del Algoritmo de Kruskal

El siguiente codigo implementa el algoritmo de Kruskal, que ordena todas las aristas por peso y las agrega al MST si no forman un ciclo.

Codigo en Java:

```

1  import java.util.*;
2
3  class Edge implements Comparable<Edge> {
4      int src, dest, weight;
5
6      public int compareTo(Edge other) {
7          return this.weight - other.weight;
8      }
9  }
10
11  class Subset {
12      int parent, rank;
13  }
14
15  class Kruskal {
16      int vertices, edges;
17      Edge[] edge;
18
19      // Constructor
20      Kruskal(int vertices, int edges) {
21          this.vertices = vertices;
22          this.edges = edges;
23          edge = new Edge[edges];
24          for (int i = 0; i < edges; i++) {
25              edge[i] = new Edge();
26          }
27      }
28
29      // Encuentra el conjunto al que pertenece un elemento
30      int find(Subset[] subsets, int i) {
31          if (subsets[i].parent != i) {
32              subsets[i].parent = find(subsets, subsets[i].parent);
33          }
34          return subsets[i].parent;
35      }

```

```

36
37 // Une dos subconjuntos
38 void union(Subset[] subsets, int x, int y) {
39     int xroot = find(subsets, x);
40     int yroot = find(subsets, y);
41     if (subsets[xroot].rank < subsets[yroot].rank) {
42         subsets[xroot].parent = yroot;
43     } else if (subsets[xroot].rank > subsets[yroot].rank) {
44         subsets[yroot].parent = xroot;
45     } else {
46         subsets[yroot].parent = xroot;
47         subsets[xroot].rank++;
48     }
49 }
50
51 // Ejecuta el algoritmo de Kruskal
52 void kruskalMST() {
53     Edge[] result = new Edge[vertices - 1];
54     int e = 0;
55     Arrays.sort(edge);
56
57     Subset[] subsets = new Subset[vertices];
58     for (int i = 0; i < vertices; i++) {
59         subsets[i] = new Subset();
60         subsets[i].parent = i;
61         subsets[i].rank = 0;
62     }
63
64     for (Edge nextEdge : edge) {
65         if (e == vertices - 1) break;
66
67         int x = find(subsets, nextEdge.src);
68         int y = find(subsets, nextEdge.dest);
69
70         if (x != y) {
71             result[e++] = nextEdge;
72             union(subsets, x, y);
73         }
74     }
75
76     System.out.println("Arista \tPeso");
77     for (int i = 0; i < e; i++) {
78         System.out.println(result[i].src + " - " + result[i].dest + "\t" +
79             result[i].weight);
80     }
81
82     public static void main(String[] args) {
83         int vertices = 4;
84         int edges = 5;
85         Kruskal graph = new Kruskal(vertices, edges);
86
87         graph.edge[0].src = 0;
88         graph.edge[0].dest = 1;
89         graph.edge[0].weight = 10;
90
91         graph.edge[1].src = 0;
92         graph.edge[1].dest = 2;

```

```

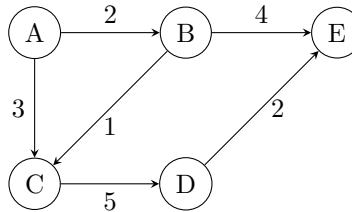
93     graph.edge[1].weight = 6;
94
95     graph.edge[2].src = 0;
96     graph.edge[2].dest = 3;
97     graph.edge[2].weight = 5;
98
99     graph.edge[3].src = 1;
100    graph.edge[3].dest = 3;
101    graph.edge[3].weight = 15;
102
103    graph.edge[4].src = 2;
104    graph.edge[4].dest = 3;
105    graph.edge[4].weight = 4;
106
107    graph.kruskalMST();
108 }
109 }

```

Ejercicio 1: Algoritmo de Prim (Difícil)

Enunciado: Dado el siguiente grafo no dirigido y ponderado, utiliza el algoritmo de Prim para encontrar el árbol de expansión mínima (MST). Incluye el análisis paso a paso y el código implementado en Java.

Grafo:



Resolución:

- **Paso 1: Inicialización.** Selecciona el vértice A como punto inicial. Llave inicial: $[0, \infty, \infty, \infty, \infty]$.
- **Paso 2: Selección de la arista de menor peso.** Selecciona $A \rightarrow B$ con peso 2.
- **Paso 3: Actualización de llaves y MST.** Llave actualizada: $[0, 2, 3, \infty, \infty]$.
- Repite el proceso hasta incluir todos los vértices en el MST.

Resultado: El MST incluye las aristas $A \rightarrow B, B \rightarrow C, D \rightarrow E$ con un costo total de 12.

Código en Java:

```

1  int [][] graph = {
2      {0, 2, 3, 0, 0},
3      {2, 0, 1, 0, 4},
4      {3, 1, 0, 5, 0},
5      {0, 0, 5, 0, 2},
6      {0, 4, 0, 2, 0}
7  };

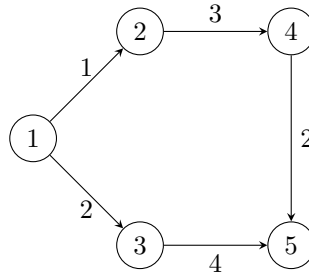
```

```
8 Prim prim = new Prim();
9 prim.primMST(graph);
```

Ejercicio 2: Algoritmo de Kruskal (Difícil)

Enunciado: Aplica el algoritmo de Kruskal al siguiente grafo para obtener el MST. Explica cada paso y proporciona el código.

Grafo:



Resolución:

- Ordena todas las aristas: 1, 2, 2, 3, 4.
- Itera sobre las aristas y agrega al MST si no forman ciclos.
- El MST final incluye las aristas $1 \rightarrow 2$, $1 \rightarrow 3$, $4 \rightarrow 5$.

Resultado: El MST tiene un costo total de 7.

Código en Java:

```
1 Kruskal graph = new Kruskal(5, 5);
2
3 graph.edge[0].src = 0;
4 graph.edge[0].dest = 1;
5 graph.edge[0].weight = 1;
6
7 graph.edge[1].src = 0;
8 graph.edge[1].dest = 2;
9 graph.edge[1].weight = 2;
10
11 graph.edge[2].src = 1;
12 graph.edge[2].dest = 3;
13 graph.edge[2].weight = 3;
14
15 graph.edge[3].src = 2;
16 graph.edge[3].dest = 4;
17 graph.edge[3].weight = 4;
18
19 graph.edge[4].src = 3;
20 graph.edge[4].dest = 4;
21 graph.edge[4].weight = 2;
22
23 graph.kruskalMST();
```